



Beyond Key-Value: Deep and Shallow Caching for Ultra-Low Latency MySQL and Postgres

Vinicius Grippa

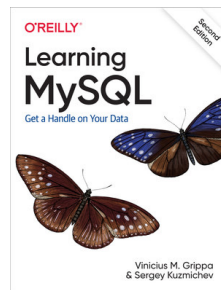
Lead Database Engineer @ Readysset

About Me



SELECT * FROM person WHERE name LIKE 'Vini';

- Lead Database Engineer at **Readysset**
- Ex-Percona working with MySQL and MongoDB
- Co-author of the book Learning MySQL
- Organizer of the MySQL community in Brazil (MUG)



COMUNIDADE
MySQL  **BR** 



Agenda



Agenda

- What is an In-Memory Database?
- The Challenges & Market Reality
- Why In-Memory DBs Excel at Query Caching
- Readysset
- Deep Caching
- Shallow Caching
- Performance



What is an In-Memory Database?

- **Core** : Data lives in **MEMORY** rather than on a Disk.
- **Sub-millisecond Speed**: Eliminates Disk I/O bottlenecks for “instant” performance.
- **High Concurrency**: Handles millions of simultaneous operations.
- **Simplified Logic**: Direct CPU-to-Memory communication.
- **Popular Examples**:
 - Readysset
 - Redis & Memcached
 - SAP HANA



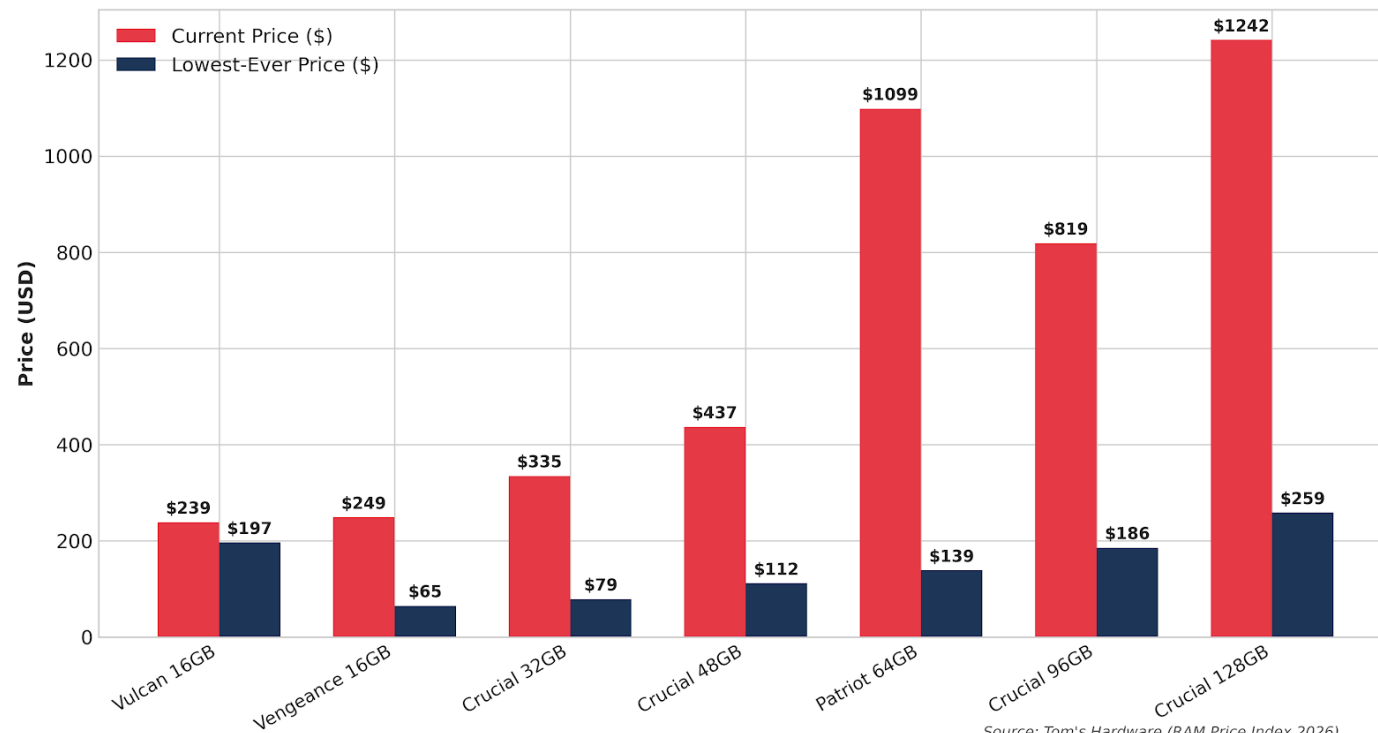
The Challenges & Market Reality

- High Cost Per GB: Significantly more expensive than traditional disk storage.
- Market Volatility: Recent DDR5 price spikes (up to 3x increases) have made scaling unpredictable.
- Capacity Caps: Total data volume is strictly limited by available physical RAM.
- Key Takeaway: You are trading budget for raw speed.



The Challenges & Market Reality

DDR5 Price Explosion: Market Comparison



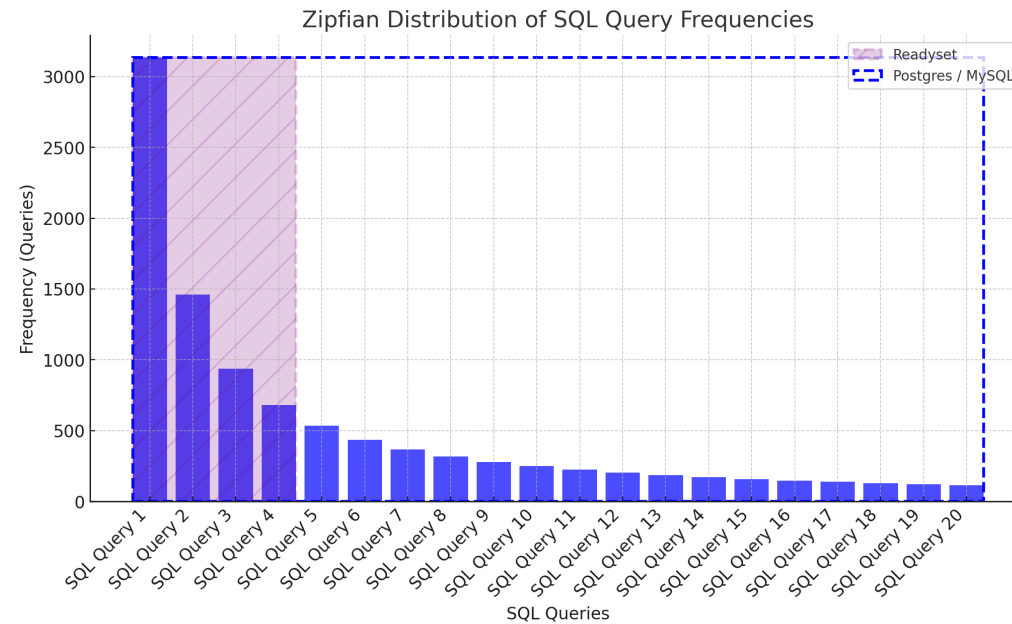
Source: Tom's Hardware (RAM Price Index 2026)



<https://www.tomshardware.com/pc-components/ram/ram-price-index-2026-lowest-price-on-ddr5-and-ddr4-memory-of-all-capacities>

The Challenges & Market Reality

- Queries often follow a Zipfian distribution
- Read heavy workloads. Infrequent changes. Eventual consistency is okay.



Why In-Memory DBs Excel at Query Caching

- Query results fit in RAM — cached data is far smaller than raw table data
- Read workloads tends to obey a Zipfian distribution
- Repeated queries = repeated work — caching eliminates redundant computation
- Sub-millisecond reads from RAM vs. milliseconds from disk on every query hit
- Yet most solutions require manual key design, TTL tuning, and custom invalidation logic. Complexity that shouldn't be the application's problem.

“The hard problem is not *storing* results in memory; it is knowing when they are no *longer* valid.”



Readysset

SQL caching engine



Readyset

- **Drop-in SQL Caching:** Sits between app and DB; no architectural changes required.
- **Sub-millisecond Performance:** Serves repeated queries from RAM to offload primary DB.
- **Auto-Maintenance:** Tracks dependencies and refreshes data automatically as it changes.
- **Simplified Scaling:** Eliminates manual cache invalidation logic across services.



Readyset x Redis

Feature	Readyset (SQL Cache)	Redis (App Cache)
Core Concept	SQL Result Cache for MySQL/Postgres	General-purpose Key-Value Store
Integration	Drop-in; minimal code changes	Requires manual Key/TTL definition
Data Sync	Dependency-aware (Auto-refresh)	Manual or TTL-based invalidation
Best For	Scaling complex SQL read queries	Session storage, rate limits, queues
Trade-off	Simplifies app logic; SQL-centric	High flexibility; adds app complexity



Deep Caching

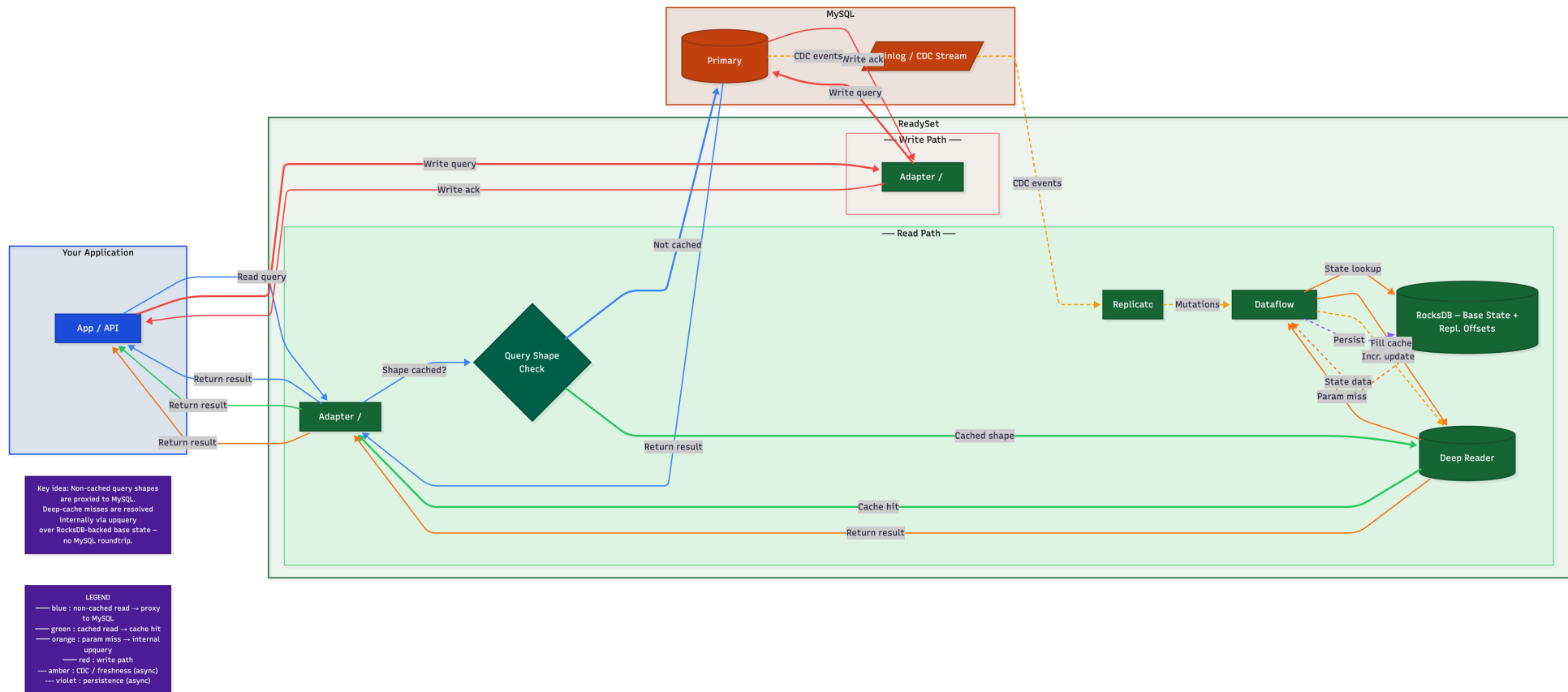


Deep Caching

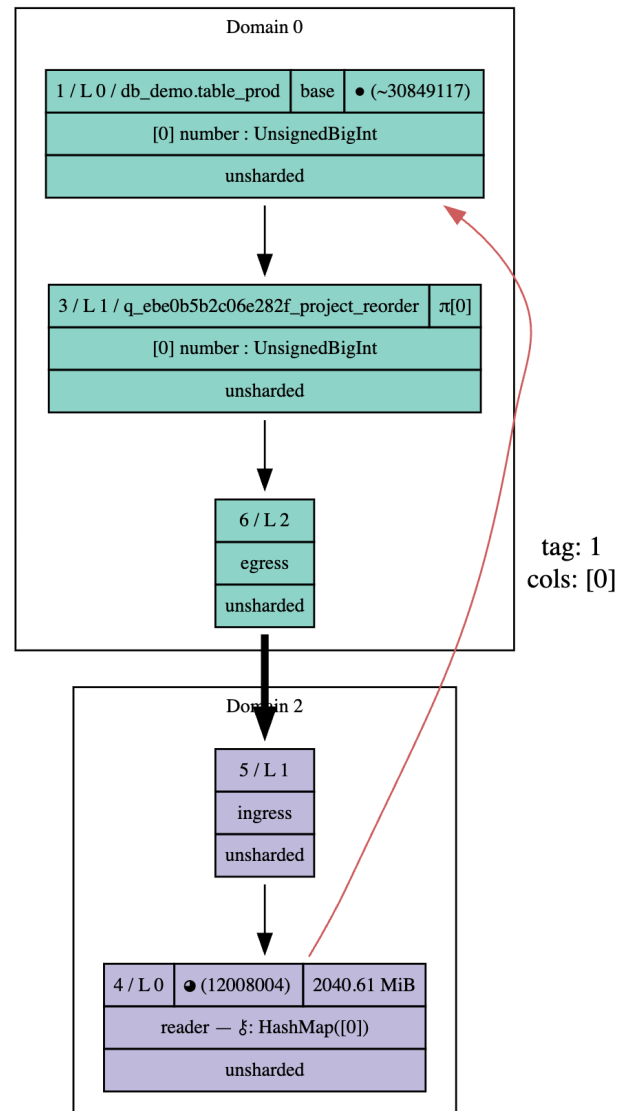
- **State storage (RocksDB):** Readysset persists replicated base-table data plus cache metadata in local RocksDB-backed state for durability and restart recovery.
- **How CDC works:** Readysset connects as a database replica and consumes the change stream continuously, instead of polling full tables.
- **How state works:** Each change is consumed from the replication stream and propagated incrementally through Readysset's dataflow graph (joins, filters, aggregates), updating only the affected portions of materialized views so cached results stay current without full recomputation.
- **How binlog updates data:** In MySQL deployments, Readysset reads binlog events (INSERT, UPDATE, DELETE, plus relevant DDL), tracks replication offsets, and incrementally updates only impacted cache state.
- **How data is served:** For deep caches, the adapter routes a query to Readysset Server if a cache exists for that query shape — since the materialized view is always kept current via replication, there is no traditional cache miss for cached queries. Queries with no cache created, unsupported queries, and all writes are forwarded to the upstream database.



Deep Caching



Deep Caching



Shallow Caching



Shallow Caching

- **TTL-based query result cache:** Shallow caching stores query results in memory for a configured time window, then refreshes when entries become stale.
- **No dataflow materialization required:** It does not build a full incremental view graph, which makes setup lighter and faster.
- **Broader query compatibility:** It can cache many queries that deep/dataflow caching may not support yet.
- **Freshness is policy-driven:** Data is refreshed by TTL and optional refresh settings (on-demand or scheduled), not by per-row CDC dependency tracking.
- **Good fit for pragmatic acceleration:** Useful when you need quick read latency improvements with simpler operational overhead, while accepting TTL-style freshness tradeoffs.

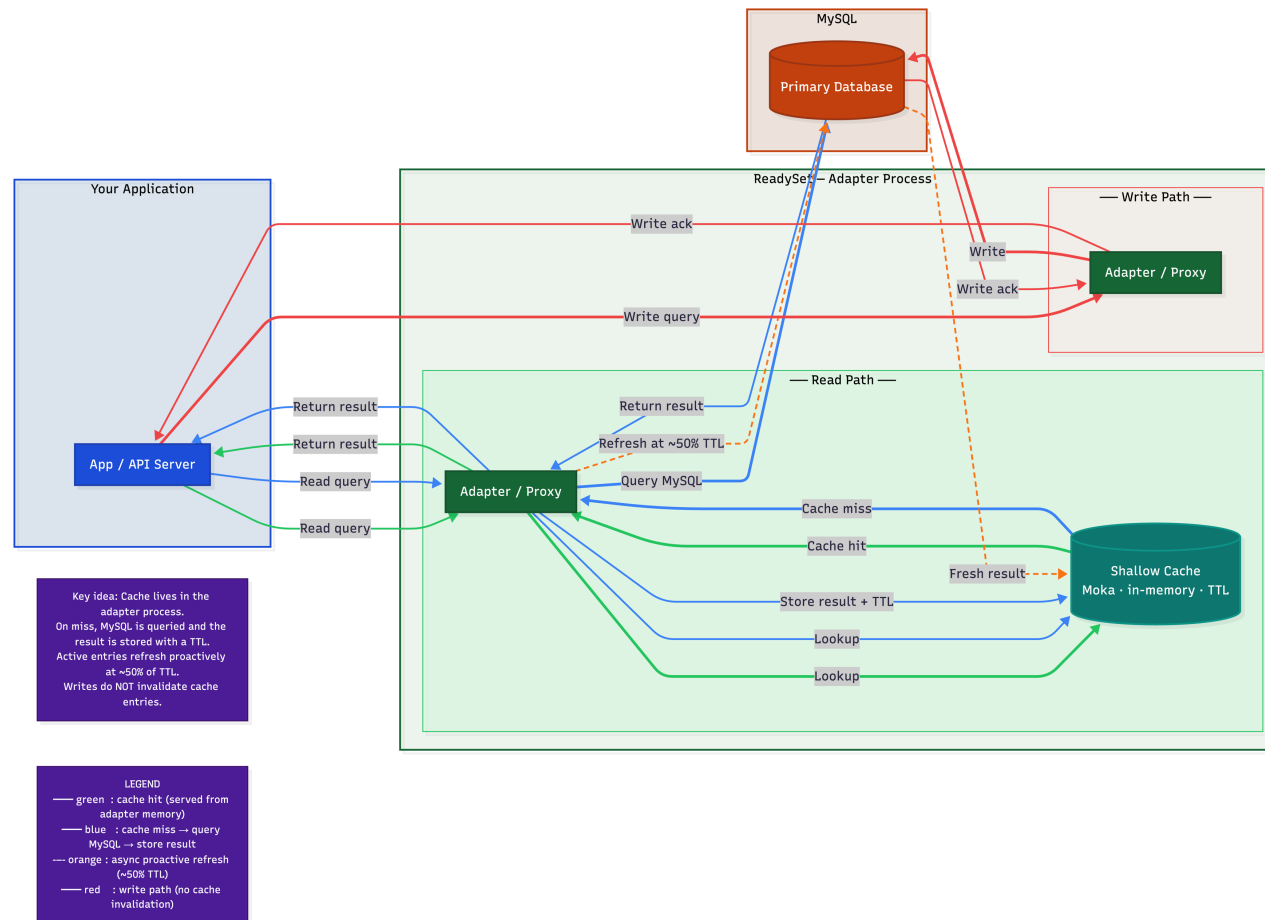


Shallow Caching vs. Redis: What Sets Readysset Apart

- **No application code changes:** Readysset manages the full cache lifecycle automatically, no manual key design, no invalidation logic
- **Thundering herd protection:** Concurrent identical requests during a cache miss are coalesced into a single upstream query, preventing a storm of duplicate DB hits
- **Atomic background refresh:** Stale entries are refreshed in the background and swapped in atomically. Users always see sub-millisecond latency, never the refresh cost
- **Intelligent TTL with on-demand refresh:** Serves the current cached value immediately while re-executing the query asynchronously behind the scenes
- **Near 100% query coverage:** Works with any SELECT, including complex aggregations, no manual mapping of queries to cache keys



Shallow Caching



Performance



Performance

```
oltp oltp_2
SELECT * FROM follows WHERE follower_id = ? LIMIT 100

oltp oltp_3
SELECT * FROM users WHERE id = ?

oltp oltp_4
SELECT * FROM likes WHERE post_id = ? LIMIT 100

oltp oltp_5
SELECT p.*, u.username FROM posts p JOIN users u ON u.id = p.user_id WHERE p.id = ?

oltp oltp_6
SELECT * FROM comments WHERE user_id = ? ORDER BY created_at DESC LIMIT 30

oltp oltp_7
SELECT * FROM follows WHERE followee_id = ? LIMIT 100

oltp oltp_8
SELECT COUNT(*) AS c FROM likes WHERE post_id = ?

oltp oltp_9
SELECT * FROM post_hashtags WHERE post_id = ?

oltp oltp_10
SELECT * FROM posts WHERE id = ?

oltp oltp_11
SELECT * FROM hashtags WHERE id = ?

analytics analytics_0
SELECT h.name, COUNT(*) AS uses FROM post_hashtags ph JOIN hashtags h ON h.id = ph.hashtag_id GROUP BY h.name ORDER BY uses DESC LIMIT 50

analytics analytics_1
SELECT user_id, COUNT(*) AS engagements FROM (SELECT user_id FROM likes UNION ALL SELECT user_id FROM comments) t GROUP BY user_id ORDER BY engagements DESC LIMIT 50

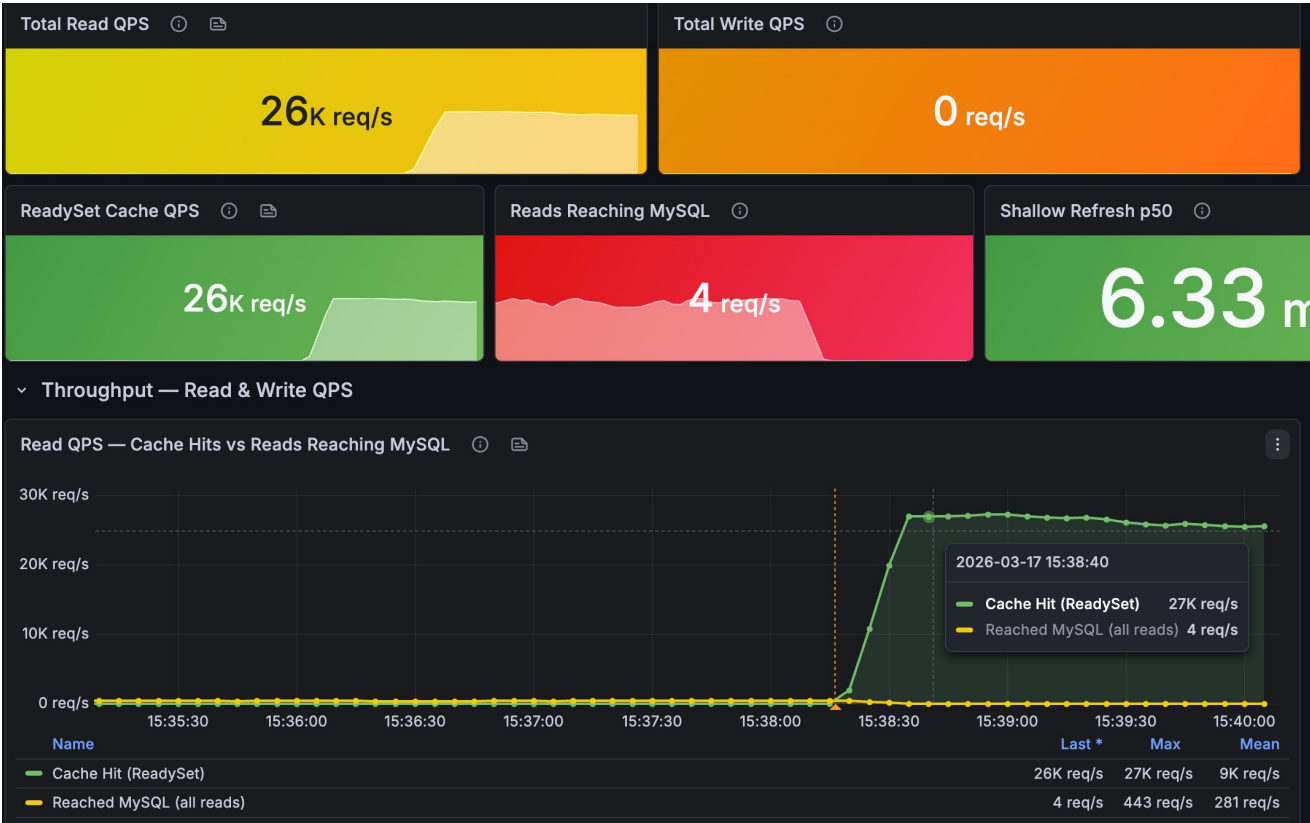
analytics analytics_2
SELECT DATE(created_at) AS d, COUNT(*) AS cnt FROM posts GROUP BY d ORDER BY d DESC LIMIT 90
```



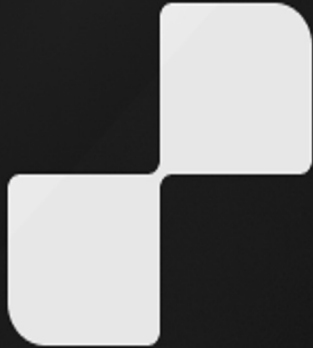
Performance



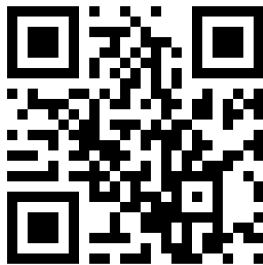
Performance



Questions?



Follow us



Thank You!

